



TRABAJOS PRÁCTICOS

Librerías útiles en Python y consejos presentación

TABLA DE CONTENIDOS

01. LIBRERÍAS ÚTILES

02. GRÁFICOS

03. CONFIGURACIÓN

04. PRESENTACIONES

05. INDUSTRIA

01

LIBRERÍAS ÚTILES



Operaciones rápidas en arrays

```
pip install numpy
```

```
import numpy as np
```

[Numpy Documentation](https://numpy.org/doc/stable/)



Librería para manejo de datos.

```
pip install pandas
```

```
import pandas as pd
```

[Pandas Documentation](https://pandas.pydata.org/pandas-docs/stable/10min.html)

De gran ayuda para el TP2 en el que hay manejar datasets grandes.



Realizar gráficos

```
pip install matplotlib
```

```
import matplotlib.pyplot as plt
```

[Matplotlib Documentation](https://matplotlib.org/)



Librería para realizar gráficos interactivos

```
pip install plotly
```

```
import plotly.express as px  
import plotly.graph_objects as go
```

[Plotly Documentación](https://plotly.com/python/)



Librería para análisis de datos.

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

[Sklearn Documentation](#)

Para el TP4 de Aprendizaje No Supervisado el modulo de decomposition para Principal component analysis (PCA) y el de preprocessing para estandarizar features.



Librería de optimización.

```
from scipy.optimize import minimize
```

[Scipy Documentation](#)

Ofrece varios métodos como Powell, BFGS, Newton, entre otros.
En particular, usamos el de Powell para el TP5 sobre Deep Learning.



ARCADE

Librería para crear videojuegos en 2D.

```
pip install arcade
```

```
import arcade
```

[Arcade Documentation](https://arcade.academy/)

Sirve para diseñar el tablero del Sokoban en el TP1 de Métodos de Búsqueda. Es fácil de aprender y de usar



Librería para crear videojuegos.

```
pip install pygame
```

```
import pygame
```

[Pygame Documentation](https://www.pygame.org/docs/)

Sirve para diseñar el tablero del Sokoban en el TP1 de Métodos de Búsqueda. Es fácil de aprender y de usar

NO UTILIZAR DEEP LEARNING FRAMEWORKS ❌

Build, train y deploy de modelos de Deep Learning

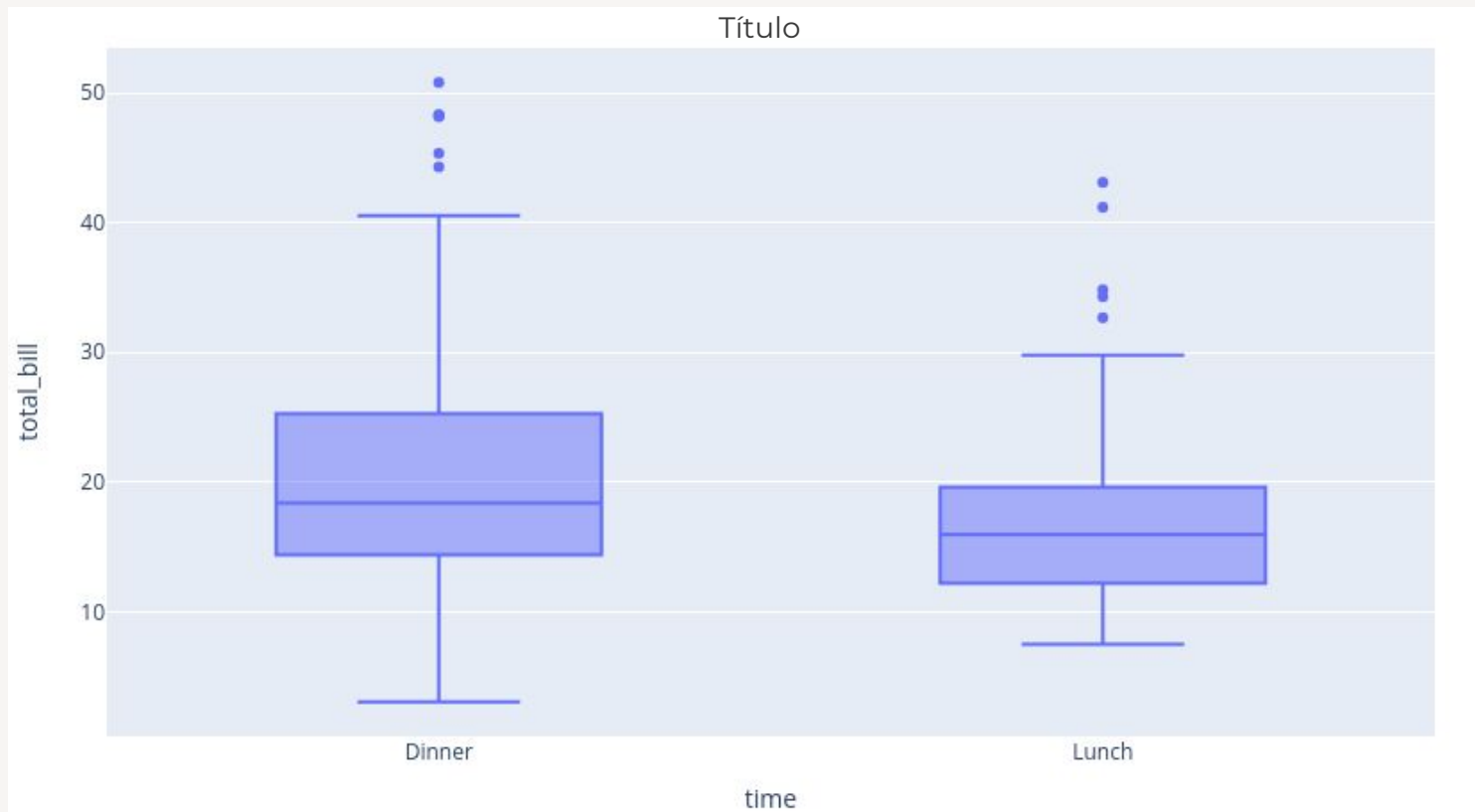


Ni ninguna otra librería relacionada con los algoritmos que vemos en la materia.

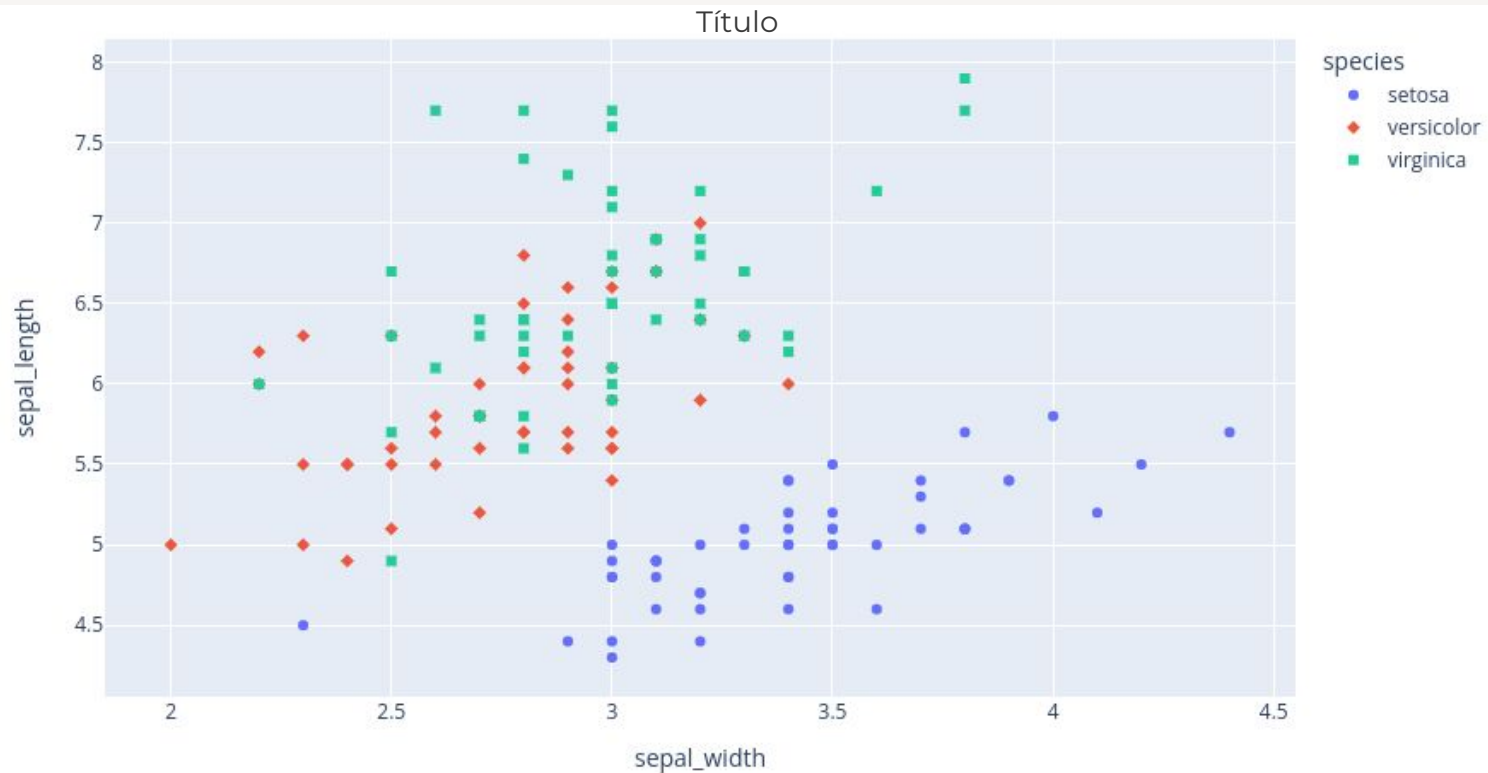
02

GRÁFICOS

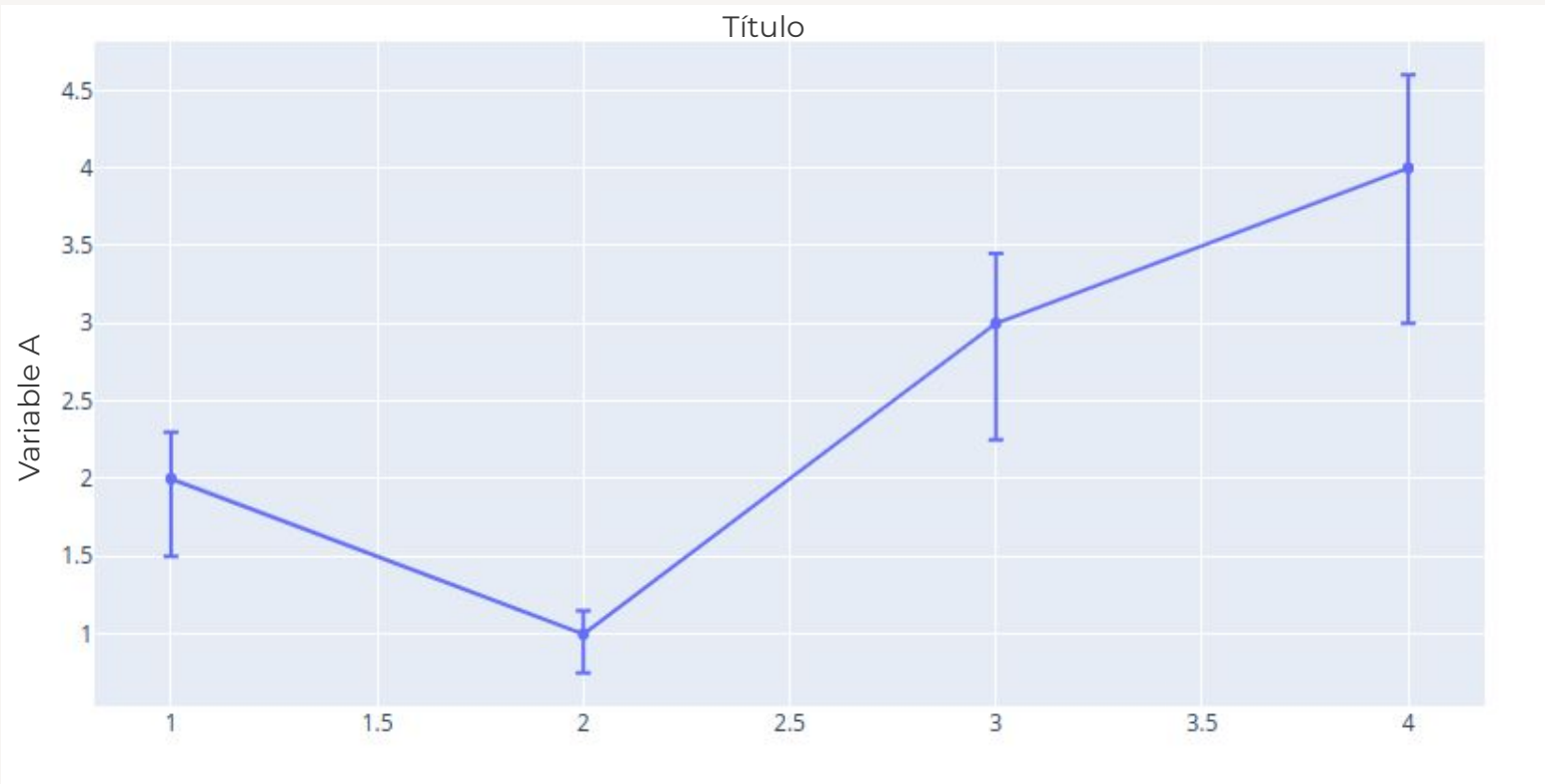
BOXPLOT



SCATTER PLOT



BARRAS DE ERROR



BARRAS DE ERROR

Gráfico de líneas con desvío estándar sombreado

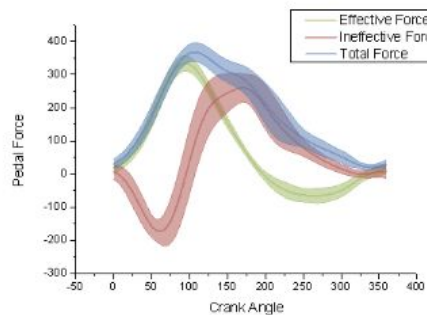
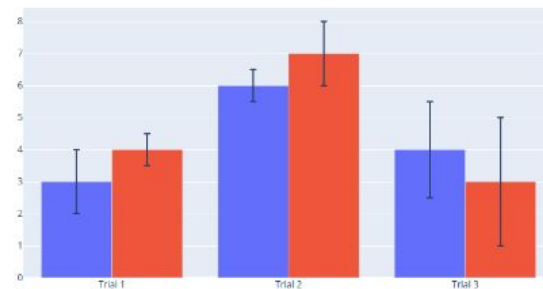


Gráfico de líneas con barras de error asimétricas



Gráfico de barras con barras de error



[PLOTLY](#)

[Seaborn](#)

03

ARCHIVO DE CONFIGURACIÓN

config.json

Example configurations: `config.json`

```
{
  "corpus": {
    "build": true,
    "name": "popular",
    "path": "../corpus/",
    "text_start": 0,
    "text_end": -1,
    "paging": 1
  },
  "postprocessing": {
    "word_cloud": true,
    "count_vectorizer": false,
    "tfidf": false,
    "word2vec": true
  }
}
```

Nota: Similar a config.yml

04

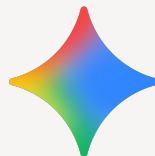
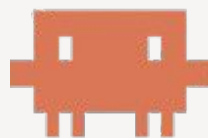
PRESENTACIONES

PRESENTACIONES

[Github SIA Tips](#)

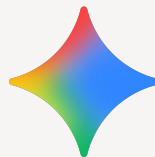
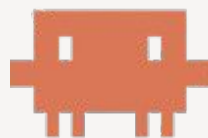
- Presentación de 25 minutos + tiempo de preguntas
- Orientada a los resultados
- Evaluación de parámetros
- Gráficos representativos de los resultados
- Promedio de ejecuciones
- Fijar un *seed* particular para reproducir resultados
- Conclusiones
- Entrega: repositorio incluyendo ppt, readme.md , config.json
- Nota individual

CODING AGENTS



1. Las decisiones de implementación deben estar **correctamente justificadas**
2. **No utilizar como caja negra:** se debe comprender todo lo que forme parte del repositorio, **mantengan el control del código**
3. Encapsulamiento de la **complejidad** dominada, no se extralimiten: usen código, ideas, consejos que ustedes sean capaces de procesar. Extender su alcance y no dejarse llevar aceptando todas las sugerencias que están por arriba de su nivel
4. Traigan luz: Usen los LLM para que les ofrezcan **claridad**
5. **Pair programming y equipo:** Teniendo acceso a LLMs se vuelve natural dividirse el trabajo y que luego nadie entienda nada de lo que se está haciendo, siendo imposible integrarlo.
6. **Explicabilidad:** Relacionarlo con la teoría y la lógica de los algoritmos que están haciendo
7. CoT: Pidan resolución **paso a paso**, cambio a cambio, para poder mantener el control y capturar que está ocurriendo.

CODING AGENTS



8. **Unit test:** Promptear incluyendo que tipo de tests se tienen que incluir
9. **Unhappy paths:** prompteen también incluyendo las situaciones bordes y
10. **Snippets:** ofrezcan ejemplos de USTEDES ya armados que los LLMs puedan tomar como base para forzar ese estilo
11. Mantengan el total control de los cambios: **no admitan cambios masivos** de muchos archivos/clases etc.
12. **Confíen** profundamente **en** lo que **ustedes** hacen: vean al agente como alguien que los desafía y demanden pequeños cambios y justificaciones
13. **Responsabilidad:** Ante cualquier problema o duda que surja del código, somos nosotros quien debemos justificar su accionar. Es inaceptable el desconocimiento del código que uno commitea
14. **Correcto uso:** aprovechar contexto con /compact o utilizar /plan mode